

RV32I CPU

Design Specification

Three-stage processor · RTL interface and programming contract

Document	RV32I CPU — Design Specification
Document type	Design specification
Subject	Three-stage processor · RTL interface and programming contract
Revision	2.2
Date	23 September 2026
Author	Bunea Cosmin-Andrei
Status	Released for review

Contents

1	Introduction	2
1.1	Purpose of the document	2
1.2	Validity of the document	2
1.3	Definitions of terms and abbreviations	2
1.4	Relationship with other documents	2
1.5	Notes on the notation used	3
2	Specifications and system integration	4
2.1	Requirements and integration	4
3	Overview of overall structure	5
3.1	Pipeline and control contract	6
3.2	Instruction groups	6
3.3	Branch prediction	6
3.4	Trap entry, return and competing events	7
3.5	Stall, forwarding and flush	7
3.6	Independent memory ports	8
4	Interfaces	9
4.1	Hardware interface	9
4.1.1	Reset contract	11
4.1.2	Instruction AXI4-Lite interface	11
4.1.3	Data AXI4-Lite interface	12
4.1.4	Interrupts and redirects	13
4.2	Software interface	14
4.2.1	CSR fields: status and interrupt enables	15
4.2.2	CSR fields: trap vector and return	15
4.2.3	CSR fields: causes and fault payload	16
4.2.4	CSR fields: identity and capability	17
4.2.5	CSR fields: counters and events	17
4.2.6	Programming sequence	18
5	Reuse and reusability	20
6	Design of submodules	21
6.1	rv32i_fetch_unit	21
6.2	rv32i_branch_predictor	22
6.3	rv32i_decode	23
6.4	rv32i_control	23
6.5	rv32i_imm_gen	24
6.6	rv32i_regfile	25
6.7	rv32i_alu_top	25
6.8	rv32i_alu	26
6.9	rv32i_branch_unit	27
6.10	rv32i_lsu	27
6.11	rv32i_exception_unit	29
6.12	rv32i_csr_file	30
6.13	rv32i_hazard_unit	31
6.14	rv32i_writeback_mux	32
7	List of bugs and workarounds	33
8	Annex	34
8.1	Shared Verilog definitions	34

1 Introduction

1.1 Purpose of the document

This specification describes the implemented RTL, programming model and integration contracts of the RV32I three-stage CPU. It is intended for hardware designers, verification engineers and software developers integrating the block.

It defines the behaviour that the RTL must show at its interfaces.

1.2 Validity of the document

The documented configuration is the RTL in `cpu/hdl` and the associated CPU/PIC and SoC simulation environments. Interface adaptations from the supplied briefs are identified in the integration section.

Revision 2.2 of 23 September 2026 describes the delivered state of that RTL.

1.3 Definitions of terms and abbreviations

Term	Meaning
AXI4-Lite	Single-beat AXI read/write channels.
BHT / BTB	Branch direction / target tables.
CSR	Control and status register.
DUT	Device under test.
EOI	End of interrupt; completes one service level.
IRQ	Interrupt request.
LSU	Load/store unit.
NBA	Nonblocking-assignment update region in simulation.
PIC	Programmable interrupt controller.
RAS	Return-address stack.
S1 / S2 / S3	Fetch / decode-execute / writeback.
SVA	SystemVerilog Assertions.

1.4 Relationship with other documents

The documents below are the ones this specification is derived from or read with.

Document	Version / date	Author	Relationship
RV32I 3-Stage Pipeline CPU	Draft, 1 July 2026	D. Borzasi, Siemens	Functional requirements; see section 2.
Spec table of contents and templates	2 September 2026	D. Borzasi, Siemens	Document structure and verification-plan template.
Verification Specification — CPU	Rev. 2.2, 23 September 2026	C.-A. Bunea	Companion document.

Document	Version / date	Author	Relationship
Design Specification — PIC	Rev. 2.2, 23 September 2026	C.-A. Bunea	Other side of the CPU–PIC interrupt link.
RISC-V Unprivileged ISA	20191213	RISC-V International	Instruction semantics.
RISC-V Privileged Architecture	20211203	RISC-V International	Machine-mode CSRs and traps.
AMBA AXI and ACE Protocol Specification	IHI0022E	Arm	AXI4-Lite ports.
IEEE Std 1364 (Verilog)	2005	IEEE	RTL language.

1.5 Notes on the notation used

Signal and register names follow the RTL. Addresses and waveform bus values are hexadecimal; time is in ns and widths are in bits. Register offsets are byte offsets; CSR addresses are instruction-encoded indices.

Notation	Meaning
Fixed-width name	A Verilog module, port, file or script name, spelled as in the source.
Signal suffix <code>_i</code> / <code>_o</code>	Module input / output, seen from inside that module.
<code>[a:b]</code>	Bit range, most significant bit first.
<code>S1</code> / <code>S2</code> / <code>S3</code>	Pipeline stage names used throughout the document.
<code>posedge +1 ns</code>	A testbench stimulus time, one nanosecond after the sampling edge.

2 Specifications and system integration

2.1 Requirements and integration

The table sets each requirement of the brief RV32I 3-Stage Pipeline CPU against the contract implemented in `rv32i_cpu_top`, including the points where the two differ. In the SoC, the instruction port reaches the instruction memory, the data port reaches data memory, the SRAM and the peripheral registers through the SoC decoder, and the interrupt link connects directly to the PIC; the address map is in `soc/README.md`.

Topic	Initial requirement	Implemented contract
Pipeline	Three stages: fetch; decode + execute; writeback.	S1 / S2 / S3; S2 waits for the complete data transaction.
ISA	RV32I integer instructions; no multiply/divide.	RV32I, machine-mode CSRs, traps, MRET and WFI.
Prediction	One direction bit per branch; structure to be defined.	Tagged direct-mapped BTB/BHT; 128 entries by default; optional return stack.
Memory	Independent AXI4-Lite instruction/data masters.	Read-only instruction port; read/write data port; 32-bit buses.
Exceptions	Define causes, entry, return and flush precedence.	Precise instruction-boundary traps; faulting instruction does not commit.
Register file	32×32 bits, two reads, one write; <code>x0 = 0</code> .	Combinational reads; S3 write; all GPRs reset to zero.
Reset / target	Active-low asynchronous reset; behavioural RTL simulation.	posedge <code>clk_i</code> ; asynchronous assertion of <code>rst_n_i</code> ; no synthesis result claimed.
IRQ integration	CPU brief: 8 requests, 3-bit ID, 8-bit acknowledgement.	Implemented PIC contract: 16 sources, scalar request/claim, 4-bit ID and EOI.

3 Overview of overall structure

Follow the instruction from S1 through IF/DX to S2, then through DX/WB to S3. The lower paths carry writeback forwarding, execution status, predictor updates and redirects; the LSU completes a memory access before S2 advances.

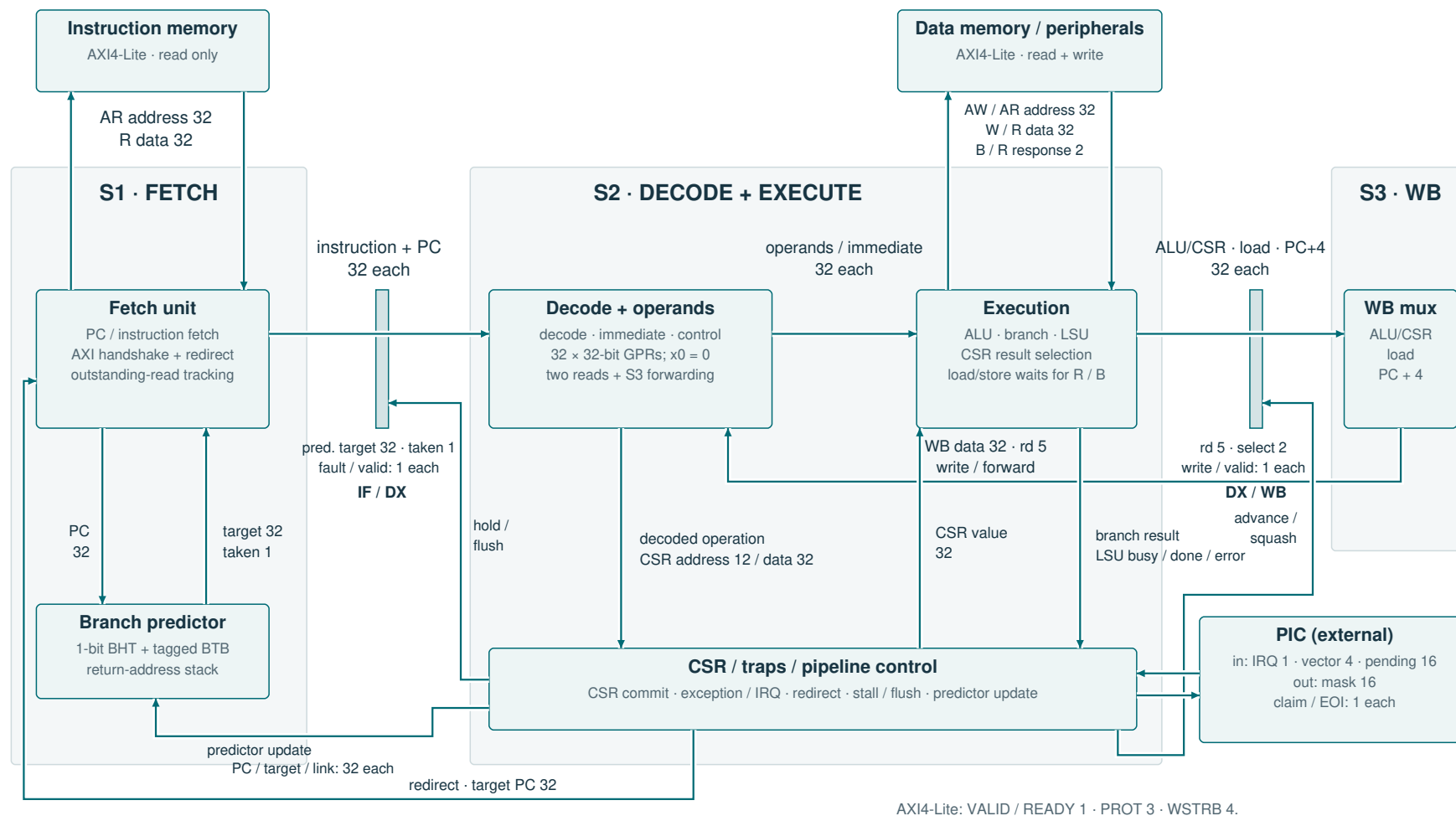


Figure 1: Three stages with pipeline boundaries, data paths and essential control feedback; widths are in bits. PIC and memories are external.

3.1 Pipeline and control contract

Each stage owns one part of an instruction, and only S2 decides when an instruction is finished. The table states what each stage is responsible for, and which redirect wins when several are requested in the same cycle.

Stage / control	Behaviour
S1	One outstanding instruction read; predictor selects next PC; wrong-path responses are drained and discarded.
S2	Decode, execute and complete memory access. CSR side effects / instruction retirement occur at completion.
S3	Write result to the destination GPR. Forward this value to dependent S2 operands.
Redirect priority	Synchronous trap / accepted interrupt, then MRET, then branch correction; redirect requires S2 to advance.

3.2 Instruction groups

Integer instructions execute in order. Loads and stores complete in S2; CSR effects also commit there, while GPR writeback occurs in S3.

Group	Implemented instructions / behavior
Arithmetic / upper immediate	ADD, ADDI, SUB, LUI, AUIPC.
Logic	AND/ANDI, OR/ORI, XOR/XORI.
Shift	SLL/SLLI, SRL/SRLI, SRA/SRAI.
Compare	SLT/SLTI, SLTU/SLTIU.
Control transfer	BEQ, BNE, BLT, BGE, BLTU, BGEU, JAL and JALR.
Load / store	LB, LH, LW, LBU, LHU; SB, SH, SW. Naturally aligned accesses only.
System	ECALL, EBREAK, FENCE as a no-op; CSRRW/CSRWS/CSRRC and immediate variants; MRET and WFI.
Unsupported encoding	Illegal-instruction exception, including M/A/C instructions and FENCE.I.

3.3 Branch prediction

Prediction chooses the next fetch address; S2 remains responsible for resolving the instruction and committing any predictor update.

Decision	Behaviour
Table and mapping	BP_ENTRIES=128 by default. A direct-mapped entry uses PC[IDX_W+1:2], with IDX_W=log2(BP_ENTRIES); upper PC bits form the tag. An alias replaces that indexed entry.
Unseen branch	Invalid entry or tag miss predicts not taken, selecting PC+4. Each valid entry stores one direction bit and a 32-bit target.
Misprediction	At S2, compare predicted direction with the resolved direction; for a taken transfer also compare targets. A mismatch redirects fetch when S2 advances.

Decision	Behaviour
Update point	A live, non-faulting branch/jump updates direction, tag and target at S2 completion, including the correcting edge. Wrong-path or trapping instructions do not train the table.
Trap entry / return	BTB/BHT and RAS are preserved across traps and MRET; reset clears them.
Call / return	RAS_DEPTH=8 by default; zero disables the stack. JAL/JALR hints use x1/x5. Return entries use the stack top, falling back to the BTB target when empty. Full pushes replace the oldest entry; empty pops have no effect.

3.4 Trap entry, return and competing events

Trap acceptance cancels the current S2 instruction and redirects younger fetch work. Older S3 work can still complete; the saved PC identifies where software must resume.

Decision	Behaviour
Entry edge	When trap_take and s2_advance are true, save mepc/mcause/mtval, copy MIE to MPIE, clear MIE, invalidate the incoming IF/DX entry and suppress the S2-to-S3 valid bit.
Return PC	Synchronous exceptions save the faulting instruction PC. An ordinary interrupt saves the preempted S2 PC. An interrupt accepted at WFI saves PC+4; software advances mepc when it elects to skip a synchronous fault.
Competing redirect	An eligible interrupt preempts the current instruction before synchronous exception evaluation. Otherwise a synchronous exception wins over MRET and branch correction. Redirect requires S2 completion.
Data transfer in progress	lsu_active prevents IRQ acceptance until the outstanding access completes. An error response causes the load/store access-fault exception at completion.
Nested service	Trap entry clears MIE. Software must save trap CSRs and handler context before enabling nested interrupts. A 16-level trap-kind stack distinguishes interrupt returns from exception returns; software must keep combined depth at or below 16.
MRET	Restore MIE from MPIE, set MPIE, redirect to mepc and pop one trap level. Emit EOI only if the popped level was an interrupt; cpu_in_trap remains high while any level is open.
MIE write versus IRQ	Arbitration uses the current MIE value. With MIE=0, a CSR write enabling it completes before later acceptance. With MIE=1 and an eligible offer, IRQ preempts the CSR instruction; that instruction can execute after return.
WFI	Hold S2 until a locally enabled offer exists. Wakeup does not require global MIE; entering the interrupt handler does.

3.5 Stall, forwarding and flush

Centralized pipeline control decides advancement and invalidation. A stalled memory instruction holds its operands in S2 while S3 receives bubbles after older work completes.

Decision	Behaviour
Fetch unavailable	No fetched instruction inserts an IF/DX bubble when S2 can advance. Fetch tracks its own address/response and may retain one returned instruction while S2 is stalled.
Data access / WFI	lsu_busy or wfi_wait holds IF/DX. S3 valid is cleared while S2 is held, preventing repeated writeback.
Load-use dependency	S2 waits for the load response. Once the load reaches S3, its result forwards to either matching S2 operand; no extra load-use bubble is needed. x0 is excluded from forwarding.
Stall and redirect	A redirect is gated by s2_advance, so the active data access completes first. At the advancing edge, trap/MRET/correction determines the new fetch address and IF/DX receives a bubble.
Wrong-path fetch	An accepted AR cannot be cancelled. An outstanding wrong-path response is drained and discarded; it cannot create a GPR write or instruction-access exception.
Pipeline registers	IF/DX stores instruction, PC, prediction and fault with valid. DX/WB stores the destination and result choices with valid. Reset clears valid bits and initializes the instruction to ADDI x0,x0,0; invalid payloads cannot execute.

3.6 Independent memory ports

Instruction fetch and the LSU have separate AXI channel state. They can progress concurrently; the pipeline consumes their results in order.

Decision	Behaviour
Both ports stalled	Hold each VALID and payload until its own READY. S2 retains the data operation; a fetched instruction can be held until S2 becomes available.
Read / write errors	Instruction RRESP[1] becomes instruction-access fault (cause 1). Data RRESP[1]/BRESP[1] becomes load/store access fault (5/7); the faulting instruction cannot write its destination.
Future AXI4 integration	An external Lite-to-Full wrapper can emit single-beat transactions with fixed ID, LEN=0, SIZE=2 and no outstanding reordering. Bursts or multiple outstanding instructions would require new request tracking and verification; the current CPU ports remain AXI4-Lite.

4 Interfaces

4.1 Hardware interface

The two memory ports and the PIC link are independent interfaces. All ports are digital and synchronous to the rising edge of `clk_i`; `rst_n_i` is the only asynchronous input.

Parameter	Default	Meaning / valid setting
RESET_PC	32'h0000_0000	Word-aligned first fetch address.
HART_ID	32'd0	Value returned by <code>mhartid</code> .
BP_ENTRIES	128	BTB/BHT entries; power of two, at least 2.
RAS_DEPTH	8	Return-address stack entries; zero disables it.

Instructions, addresses and data are 32 bits and GPR indices 5 bits. The interrupt link has 16 mask and pending bits and a 4-bit source ID. `BP_ENTRIES` and `RAS_DEPTH` set internal capacity only; no port width depends on a parameter.

Signal	Dir.	Width (bits)	Meaning
<code>clk_i</code>	input	1	Rising-edge clock input.
<code>rst_n_i</code>	input	1	Asynchronous reset, active low.
<code>ibus_axi_araddr_o</code>	output	32	Read byte address.
<code>ibus_axi_arprot_o</code>	output	3	Read protection attributes.
<code>ibus_axi_arvalid_o</code>	output	1	Read address is valid; hold until handshake.
<code>ibus_axi_arready_i</code>	input	1	Receiver can accept the read address.
<code>ibus_axi_rdata_i</code>	input	32	Read payload.
<code>ibus_axi_rresp_i</code>	input	2	Read response: OKAY=00, SLVERR=10, DECERR=11.
<code>ibus_axi_rvalid_i</code>	input	1	Read response is valid; hold until handshake.
<code>ibus_axi_rready_o</code>	output	1	Receiver can accept the read response.
<code>dbus_axi_awaddr_o</code>	output	32	Write byte address.
<code>dbus_axi_awprot_o</code>	output	3	Write protection attributes.
<code>dbus_axi_awvalid_o</code>	output	1	Write address is valid; hold until handshake.
<code>dbus_axi_awready_i</code>	input	1	Receiver can accept the write address.
<code>dbus_axi_wdata_o</code>	output	32	Write payload.
<code>dbus_axi_wstrb_o</code>	output	4	One enable bit per write byte.
<code>dbus_axi_wvalid_o</code>	output	1	Write data is valid; hold until handshake.
<code>dbus_axi_wready_i</code>	input	1	Receiver can accept the write data.
<code>dbus_axi_bresp_i</code>	input	2	Write response: OKAY=00, SLVERR=10, DECERR=11.
<code>dbus_axi_bvalid_i</code>	input	1	Write response is valid; hold until handshake.
<code>dbus_axi_bready_o</code>	output	1	Receiver can accept the write response.
<code>dbus_axi_araddr_o</code>	output	32	Read byte address.

Signal	Dir.	Width (bits)	Meaning
dbus_axi_arprot_o	output	3	Read protection attributes.
dbus_axi_arvalid_o	output	1	Read address is valid; hold until handshake.
dbus_axi_arready_i	input	1	Receiver can accept the read address.
dbus_axi_rdata_i	input	32	Read payload.
dbus_axi_rresp_i	input	2	Read response: OKAY=00, SLVERR=10, DECERR=11.
dbus_axi_rvalid_i	input	1	Read response is valid; hold until handshake.
dbus_axi_rready_o	output	1	Receiver can accept the read response.
cpu_irq_i	input	1	Prioritized interrupt request from PIC.
cpu_irq_vec_i	input	4	Source ID associated with cpu_irq_i.
irq_pending_i	input	16	Enabled pending sources from PIC; exposed in mip[31:16].
irq_mask_o	output	16	mie[31:16], sent to the PIC resolver.
cpu_irq_ack_o	output	1	Registered one-cycle claim to PIC.
cpu_irq_eoi_o	output	1	One-cycle EOI on return from an interrupt.
cpu_in_trap_o	output	1	CPU has an active trap context.

4.1.1 Reset contract

Reset must leave the core in a state software can start from, without needing a clock edge to get there. The table lists what reset forces; the waveform below shows it interrupting a fetch that is already in progress.

Item	Required behaviour
Clock	Sequential state updates on posedge clk_i.
Assertion	Asynchronous active-low rst_n_i; no clock edge required for assertion.
Outputs	AXI request VALID signals, IRQ claim/EOI and trap-active state are inactive.
Architectural state	GPRs=0; PC=RESET_PC; mstatus=0x1800; other stored CSRs/counters=0, except documented fixed values.
mip	Combinational pending input, not a reset storage element. Zero requires irq_pending_i=0.
Integration	Distribute reset to CPU and bus responders. Release it with the system reset controller; release between clock edges is also supported.

Reset is asserted while an instruction request is waiting. ARVALID clears at the reset event, before the next rising edge; after release the fetch request resumes from RESET_PC.

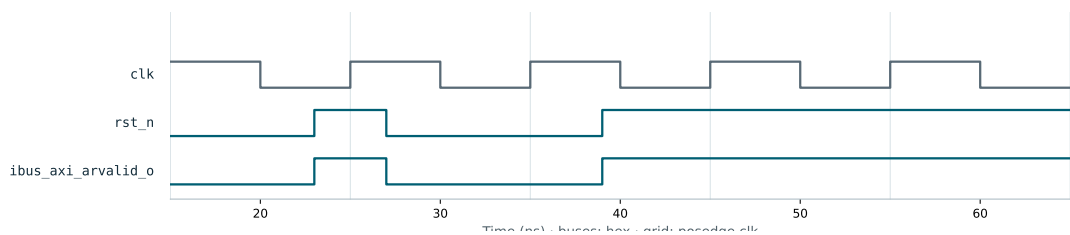


Figure 2: Instruction request interrupted by asynchronous reset at 27 ns.

4.1.2 Instruction AXI4-Lite interface

ARVALID remains asserted until the instruction address is accepted. The R handshake supplies the instruction to IF/DX; the address trace shows the next sequential fetch.

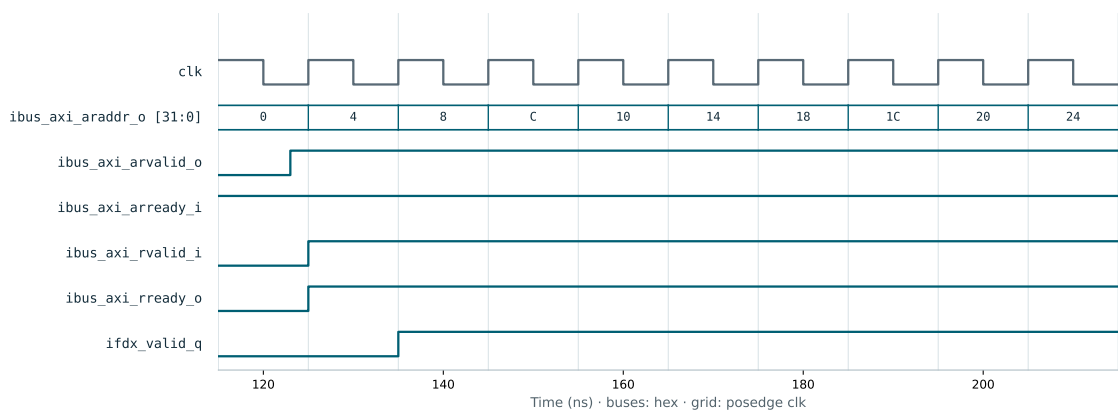


Figure 3: Instruction fetch after rst_n release at 123 ns.

Property	Contract
Channels	AR/R only; fixed 32-bit address and data.
Handshake	Transfer at posedge when VALID and READY are both high; retain VALID and payload while stalled.
Outstanding requests	At most one; issuing the next AR may overlap completion of the prior R.
Protection	ARPROT=111: instruction, non-secure, privileged.
Errors	SLVERR/DECERR become an instruction-access fault in S2. A wrong-path fault is discarded.
Redirect	An accepted request cannot be cancelled. Retain its route / drain the response before using the new path.

4.1.3 Data AXI4-Lite interface

The load holds S2 while its request or response is pending. At the RVALID/RREADY handshake, the data is available and s2_advance permits the instruction to enter writeback.

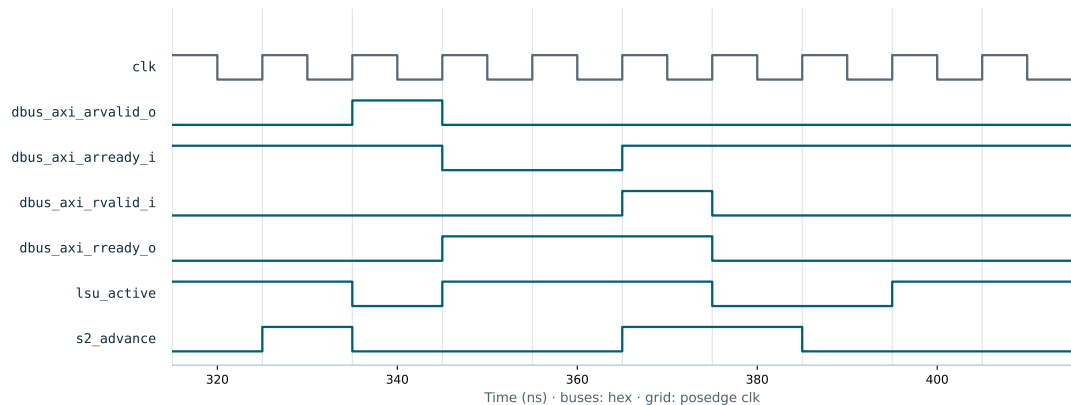


Figure 4: Load: lsu_active holds S2 until the data response completes.

AW and W are accepted independently. The store completes only when the B response is accepted, so an earlier address or data handshake alone does not release the instruction.

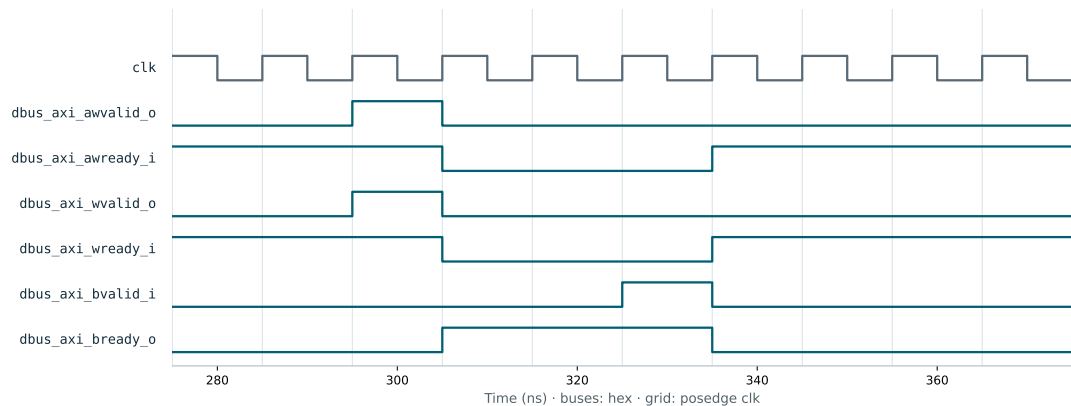


Figure 5: Store: address and write data are accepted independently; B completes the store.

Property	Contract
Transactions	One load or store outstanding; byte, halfword and word accesses.
Byte lanes	WSTRB selects bytes; reads extract/sign-extend according to the instruction.
Misalignment	Detected before bus issue; misaligned accesses trap.
Errors	SLVERR/DECERR produce load/store access faults; destination is not committed.
Protection	AWPROT/ARPROT=011: data, non-secure, privileged.

4.1.4 Interrupts and redirects

The interrupt arrives while the LSU is active. Marker 2 identifies the accepted load response; marker 3 shows the registered claim and trap-active state at the next instruction boundary.

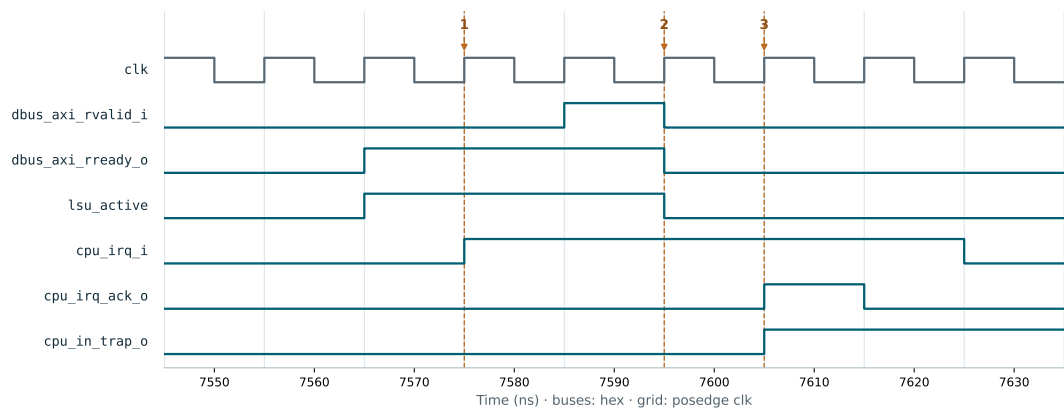


Figure 6: Interrupt claim: `cpu_irq_ack_o` is a full-cycle registered pulse.

The resolved branch differs from the prediction, causing redirect. IF/DX is invalidated so younger work cannot commit; an outstanding fetch response still completes its AXI transaction.

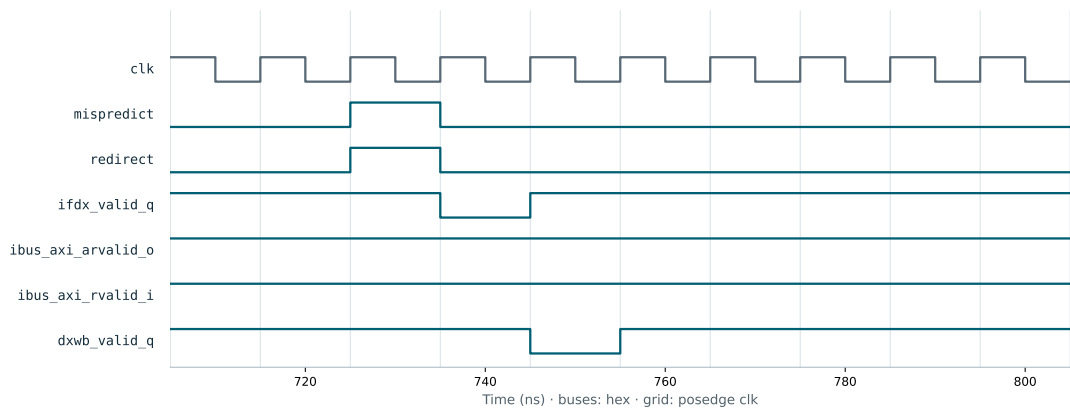


Figure 7: Branch correction: redirect invalidates younger work.

Event	Contract
Interrupt eligibility	mstatus.MIE and the enabled source must allow it; acceptance waits for an instruction boundary, including completion of an active LSU transfer.
Cause	External source n uses $mcause=0x80000000 \mid (16+n)$.

Event	Contract
Claim / EOI	Claim is sent after the offer is sampled. EOI is sent when returning from an interrupt; a synchronous exception return does not pop the PIC.
Nested software	Save mepc, mstatus and handler context before enabling nested interrupts; restore before MRET.

4.2 Software interface

The CSR file is the only software-visible interface of the core, and it is reached through CSR instructions rather than through the AXI ports. The map lists every implemented address with its access class and reset value; the subsections that follow give the fields of each register.

Address	Name	Access type	Reset	Description
0x300	mstatus	RW / fixed	0x00001800	Global interrupt state.
0x301	misa	WARL / fixed	0x40000100	RV32I capability.
0x304	mie	RW	0	Interrupt enables [31:16].
0x305	mtvec	RW / WARL	0	Trap base and mode.
0x323–0x327	mhpmevent3..7	WARL / fixed	1..5	Fixed performance-event selectors.
0x328–0x33F	mhpmevent8..31	WARL / fixed	0	Unused event selectors.
0x340	mscratch	RW	0	Software scratch word.
0x341	mepc	RW / aligned	0	Trap return PC.
0x342	mcause	RW	0	Trap cause.
0x343	mtval	RW	0	Fault payload.
0x344	mip	RO	input-dependent	irq_pending_i in [31:16].
0xB00 / 0xB80	mcycle / mcycleh	RW	0	64-bit cycle count.
0xB02 / 0xB82	minstret / minstreth	RW	0	64-bit completed-instruction count.
0xB03–0xB07 / 0xB83–0xB87	mhpmcounter3..7 / high	RW	0	64-bit event counts.
0xB08–0xB1F / 0xB88–0xB9F	mhpmcounter8..31 / high	WARL / fixed	0	Unused counters; writes ignored.
0xF11–0xF13	mvendorid / marchid / mimpid	RO	0	Identification values.
0xF14	mhartid	RO	HART_ID	Instance identifier.

Access rule	Behaviour
Unknown address / read-only write	Illegal-instruction exception; CSR addresses are not memory-mapped AXI addresses.

Access rule	Behaviour
CSRRW / CSRRS / CSRRC	Replace / set / clear; immediate variants use zero-extended zimm. Zero-source CSRRS/CSRRC do not write.
Unimplemented inhibit	0x320 (mcountinhibit) is absent and traps.

4.2.1 CSR fields: status and interrupt enables

mstatus — 0x300

Bits	Field	Access type	Reset	Description
[31:13], [10:8], [6:4], [2:0]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[12:11]	MPP	fixed	3	Machine mode only.
[7]	MPIE	RW + HW	0	Saved MIE; MRET sets it to 1.
[3]	MIE	RW + HW	0	Global enable; trap clears it; MRET restores MPIE.

mie — 0x304

Bits	Field	Access type	Reset	Description
[31:16]	ENABLE	RW	0	16 source-enable bits; exported as irq_mask_o.
[15:0]	Reserved	RAZ/WI	0	Reads zero; writes ignored.

mip — 0x344

Bits	Field	Access type	Reset	Description
[31:16]	PENDING	RO	input	Reflects irq_pending_i; zero when the connected PIC is reset.
[15:0]	Reserved	RO	0	Reads zero.

4.2.2 CSR fields: trap vector and return

mtvec — 0x305

Bits	Field	Access type	Reset	Description
[31:2]	BASE	RW	0	Word-aligned handler base.
[1:0]	MODE	RW / WARL	0	0 direct; 1 vectored. Values 2/3 read back as direct.

Trap type	Entry address
Synchronous exception	BASE in both modes.
External interrupt, direct	BASE.
External interrupt, vectored	BASE + 4 × (16 + source ID).

mepc — 0x341

Bits	Field	Access type	Reset	Description
[31:2]	RETURN_PC	RW + HW	0	Saved PC; software may update the return location.
[1:0]	Alignment	fixed	0	Reads zero; writes discarded.

mscratch — 0x340

Bits	Field	Access type	Reset	Description
[31:0]	VALUE	RW	0	Software-owned scratch word.

4.2.3 CSR fields: causes and fault payload

mcause — 0x342

Bits	Field	Access type	Reset	Description
[31]	INTERRUPT	RW + HW	0	Trap entry sets 1 for interrupt, 0 for synchronous exception.
[30:5]	Software bits	RW	0	Hardware trap entry writes zero; software writes are retained by this RTL.
[4:0]	CODE	RW + HW	0	Hardware cause code; external source n is 16+n.

mtval — 0x343

Bits	Field	Access type	Reset	Description
[31:0]	VALUE	RW + HW	0	Faulting address or illegal instruction; zero where the cause supplies no payload.

Code	Synchronous cause
0	Instruction-address misalignment.
1	Instruction-access fault.
2	Illegal instruction / invalid CSR access.

Code	Synchronous cause
3	EBREAK.
4 / 6	Load / store address misalignment.
5 / 7	Load / store access fault.
11	ECALL from machine mode.

4.2.4 CSR fields: identity and capability

misa — 0x301

Bits	Field	Access type	Reset	Description
[31:30]	MXL	fixed WARL	1	32-bit architecture; writes ignored.
[29:9], [7:0]	Other extensions	fixed WARL	0	Not advertised; writes ignored.
[8]	I	fixed WARL	1	RV32I base integer set.

mvendorid / marchid / mimpid — 0xF11 / 0xF12 / 0xF13

Bits	Field	Access type	Reset	Description
[31:0]	VALUE	RO	0	No implementation-specific identifier assigned.

mhartid — 0xF14

Bits	Field	Access type	Reset	Description
[31:0]	VALUE	RO	HART_ID	Instance parameter; use unique values for multiple harts.

4.2.5 CSR fields: counters and events

mcycle / minstret / mhpmpcounter3..7 — low and high CSR addresses in map

Bits	Field	Access type	Reset	Description
[31:0]	LOW / HIGH	RW	0	Each address accesses one half of a 64-bit counter. Explicit writes suppress that cycle's event increment and preserve the other half.

mhpmevent3..7 — 0x323–0x327

Bits	Field	Access type	Reset	Description
[31:0]	EVENT	fixed WARL	1..5	Fixed event IDs; writes accepted and ignored.

Unused HPM counters / selectors — addresses in map

Bits	Field	Access type	Reset	Description
[31:0]	VALUE	fixed WARL	0	Reads zero; writes accepted and ignored.

Counter	Event
mcycle	Every cycle.
minstret	Non-trapping instruction completes S2.
mhpmcounter3	Mispredicted control transfer.
mhpmcounter4	S2 has no valid instruction.
mhpmcounter5	Data transaction active.
mhpmcounter6	Trap entry.
mhpmcounter7	WFI sleep cycle.

A 64-bit counter is read high, low, high; the read is repeated when the two high values differ.

4.2.6 Programming sequence

The order below is the one the hardware assumes: a handler address and the masks must exist before delivery is enabled. The second table lists the limits that software written for this core has to respect.

Step	Action	Expected result
1	Set RESET_PC and provide executable instruction memory.	First fetch is word aligned.
2	Initialize stack/context and program mtvec.	Trap entry reaches a valid handler.
3	Configure PIC source type, band and INT_ENABLE.	Sources can become pending.
4	Set mie[16+n], then mstatus.MIE.	CPU can accept source n at an instruction boundary.
5	Handler services and clears the peripheral; MRET returns.	CPU generates EOI for an interrupt return.
6	For nesting, save trap CSRs/context before re-enabling MIE.	Inner handler cannot overwrite the outer return state.

Integration constraint	Consequence
No M/A/C extensions, caches or MMU	Do not emit unsupported instructions; target machine-mode RV32I with the documented CSR extensions.
Protection constant	The core does not switch bus privilege/security attributes.
Reset and source synchrony	Clock/reset must reach the full transaction path; asynchronous external IRQ lines need synchronization outside this block.
Behavioural verification target	No area, frequency, power or physical timing result is specified.

5 Reuse and reusability

The core contains no third-party component. `rv32i_alu`, `rv32i_imm_gen`, `rv32i_decode` and `rv32i_regfile` can be reused unchanged in another RV32I datapath; `rv32i_branch_predictor` is sized by `ENTRIES` and `RAS_DEPTH`. Reusing the whole core:

Task	How to use this block
Source list	Compile <code>cpu/debug/sim/rtl.f</code> from its simulation directory, or copy its RTL entries into the integrating project.
Instantiation	Instantiate <code>rv32i_cpu_top</code> and override <code>RESET_PC</code> , <code>HART_ID</code> , <code>BP_ENTRIES</code> or <code>RAS_DEPTH</code> as needed. No external signal width is parameterized.
Memory	Connect <code>ibus</code> to instruction memory and <code>dbus</code> to the memory/peripheral fabric. Honour all five data-port channels independently.
Interrupts	Connect the documented scalar request, 4-bit vector, masks, pending set and claim/EOI pair.
Tie-offs	Without interrupts: <code>cpu_irq_i=0</code> , <code>cpu_irq_vec_i=0</code> and <code>irq_pending_i=0</code> .
Verification	Run <code>regress.do</code> and <code>run_verilator.sh</code> , then test the integration map and slave latency of the target system.

6 Design of submodules

Submodule ports are synchronous to the rising edge of `clk_i` and `rst_n_i` resets asynchronously; combinational modules have neither.

6.1 rv32i_fetch_unit

Fetches instructions; retains outstanding AXI requests across redirect and discards wrong-path responses.

6.1.1 Interface description

Parameter	Default	Meaning / valid setting	
RESET_PC	32'h0000_0000	Word-aligned first fetch address.	

Signal	Dir.	Width (bits)	Meaning
<code>clk_i</code>	input	1	Rising-edge clock input.
<code>rst_n_i</code>	input	1	Asynchronous reset, active low.
<code>ibus_araddr_o</code>	output	32	Read byte address.
<code>ibus_arprot_o</code>	output	3	Read protection attributes.
<code>ibus_arvalid_o</code>	output	1	Read address is valid; hold until handshake.
<code>ibus_arready_i</code>	input	1	Receiver can accept the read address.
<code>ibus_rdata_i</code>	input	32	Read payload.
<code>ibus_rresp_i</code>	input	2	Read response: OKAY=00, SLVERR=10, DECERR=11.
<code>ibus_rvalid_i</code>	input	1	Read response is valid; hold until handshake.
<code>ibus_rready_o</code>	output	1	Receiver can accept the read response.
<code>bp_lookup_pc_o</code>	output	32	Fetch PC presented to the branch predictor.
<code>bp_pred_taken_i</code>	input	1	Predicted taken control transfer at the lookup PC.
<code>bp_pred_target_i</code>	input	32	Predicted next PC when the taken prediction is valid.
<code>redirect_i</code>	input	1	Discard younger fetch work and select <code>redirect_pc_i</code> .
<code>redirect_pc_i</code>	input	32	New fetch PC selected by trap, return or branch correction.
<code>s2_ready_i</code>	input	1	S2 can accept a new instruction into IF/DX this cycle.
<code>instr_valid_o</code>	output	1	Fetches instruction and metadata can be accepted by S2.
<code>instr_o</code>	output	32	Fetches 32-bit instruction word.
<code>instr_pc_o</code>	output	32	Byte address of the fetched instruction.
<code>pred_taken_o</code>	output	1	Tagged predictor hit with taken direction.

Signal	Dir.	Width (bits)	Meaning
pred_target_o	output	32	Predicted target address; used only when pred_taken_o is high.
fetch_fault_o	output	1	Instruction response reported an AXI error.

6.1.2 Functional description

An address request is retained until AR handshake, then inflight tracks the owed R response. A returned instruction is either consumed by S2 or retained in the holding slot. Redirect drops held data and marks an owed response for discard; the next request uses the redirected PC when the slot is free.

6.2 rv32i_branch_predictor

Predicts direction and target; learns completed control transfers; handles call/return hints.

6.2.1 Interface description

Parameter	Default	Meaning / valid setting
ENTRIES	128	Power-of-two BTB/BHT entries; at least 2.
RAS_DEPTH	8	Return-address stack entries; zero disables it.

Signal	Dir.	Width (bits)	Meaning
clk_i	input	1	Rising-edge clock input.
rst_n_i	input	1	Asynchronous reset, active low.
lookup_pc_i	input	32	PC looked up in the predictor.
pred_taken_o	output	1	Tagged predictor hit with taken direction.
pred_target_o	output	32	Predicted target address; used only when pred_taken_o is high.
update_en_i	input	1	Enable one predictor update at the rising edge.
update_pc_i	input	32	Resolved control-transfer PC.
update_taken_i	input	1	Resolved direction bit.
update_target_i	input	32	Resolved target address.
update_is_ret_i	input	1	Mark entry as a return.
update_call_i	input	1	Push return address.
update_ret_i	input	1	Pop return address.
update_link_i	input	32	Return address PC+4.

6.2.2 Functional description

The lookup is combinational and an update is committed on posedge. The prediction and update policies, including aliases, cold entries and trap preservation, are specified in section 3.3.

6.3 rv32i_decode

Extracts register indices, opcode, function fields and CSR address from the instruction.

6.3.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
instr_i	input	32	32-bit instruction word to decode or check.
opcode_o	output	7	Instruction opcode, bits [6:0].
rd_o	output	5	Destination GPR index from instruction bits [11:7].
funct3_o	output	3	Instruction function field, bits [14:12].
rs1_o	output	5	First source GPR index from instruction bits [19:15].
rs2_o	output	5	Second source GPR index from instruction bits [24:20].
funct7_o	output	7	Instruction function field, bits [31:25].

6.3.2 Functional description

Field extraction is purely combinational: opcode, funct3, funct7, the three register indices and the CSR address are sliced from fixed instruction bit positions. The module applies no legality check; an unsupported encoding is rejected by rv32i_control.

6.4 rv32i_control

Decodes instruction controls and rejects unsupported encodings.

6.4.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
opcode_i	input	7	Instruction opcode, bits [6:0].
funct3_i	input	3	Instruction function field; selects operation or memory size/sign.
funct7_i	input	7	Instruction function field; distinguishes arithmetic and shift variants.
rs1_i	input	5	First source GPR index (or immediate CSR operand).
rd_i	input	5	Destination GPR index; zero can suppress a CSR read.
imm12_i	input	12	Instruction bits [31:20]; distinguishes ECALL, EBREAK, MRET and WFI.

Signal	Dir.	Width (bits)	Meaning
RegWrite_o	output	1	Decoded instruction writes its destination register.
ALUSrc_o	output	1	Second ALU operand select: 0 register rs2, 1 immediate.
ALUOp_o	output	4	Decoded ALU operation class; encodings are in rv32i_defines.vh.
MemRead_o	output	1	Decoded instruction performs a load.
MemWrite_o	output	1	Decoded instruction performs a store.
MemtoReg_o	output	2	Writeback source select; a CSR read uses the ALU slot.
Branch_o	output	1	Decoded instruction is a conditional branch.
Jump_o	output	1	Decoded instruction is JAL or JALR.
csr_instr_o	output	1	Decoded instruction is a Zicsr CSR access.
csr_op_o	output	2	CSR operation: RW=01, RS=10, RC=11.
csr_imm_o	output	1	Immediate CSR form; the operand is the five-bit field in the rs1 position.
mret_o	output	1	Decoded instruction returns from a machine-mode trap.
ecall_o	output	1	Decoded ECALL requests an environment-call exception.
ebreak_o	output	1	Decoded EBREAK requests a breakpoint exception.
wfi_o	output	1	Decoded WFI; S2 waits for an enabled interrupt offer.
illegal_o	output	1	Encoding is not supported by the implemented instruction set.

6.4.2 Functional description

A case over the opcode, refined by funct3 and funct7, produces the control set for one instruction. An encoding that matches no supported case asserts illegal_o and leaves the write-enable and memory controls inactive, so a rejected instruction cannot change architectural state.

6.5 rv32i_imm_gen

Forms and sign-extends I/S/B/U/J immediates.

6.5.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
instr_i	input	32	32-bit instruction word to decode or check.

Signal	Dir.	Width (bits)	Meaning
imm_out_o	output	32	Immediate expanded to 32 bits for the decoded instruction format.

6.5.2 Functional description

The immediate format is selected from the opcode: I, S, B, U or J. Each form is assembled from its instruction bit fields and sign-extended to 32 bits, with the implicit zero bit added for the B and J displacements.

6.6 rv32i_regfile

Stores 32 general-purpose registers; ignores x0 writes; reads x0 as zero.

6.6.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
clk_i	input	1	Rising-edge clock input.
rst_n_i	input	1	Asynchronous reset, active low.
rs1_addr_i	input	5	First register index.
rs2_addr_i	input	5	Second register index.
rd_addr_i	input	5	Destination register index.
rd_data_i	input	32	Register write data.
RegWrite_i	input	1	Write-enable for nonzero destination register.
rs1_data_o	output	32	First register operand.
rs2_data_o	output	32	Second register operand.

6.6.2 Functional description

Both read ports are combinational, so an operand is available in the cycle it is addressed. A write commits at posedge when RegWrite_i is high and the destination is nonzero; index zero is never written and always reads as zero. Reset clears all 32 registers.

6.7 rv32i_alu_top

Selects operands and decodes the concrete ALU operation.

6.7.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
operand_a_i	input	32	First 32-bit ALU operand.
operand_b_reg_i	input	32	Second register operand, taken from rs2.
operand_b_imm_i	input	32	Decoded immediate operand.
ALUSrc_i	input	1	Second operand select: 0 register, 1 immediate.
ALUOp_i	input	4	Generic operation class from rv32i_control; encodings are in rv32i_defines.vh.
funct3_i	input	3	Instruction function field; selects operation or memory size/sign.
funct7_i	input	7	Instruction function field; distinguishes arithmetic and shift variants.
result_o	output	32	32-bit ALU result for the selected operation.

6.7.2 Functional description

The unit selects the second operand between rs2 and the immediate, then translates the generic operation class from the decoder, together with funct3 and funct7, into the concrete ALU function. It holds no state.

6.8 rv32i_alu

Computes 32-bit arithmetic, logic, shifts and signed/unsigned comparisons.

6.8.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
operand_a_i	input	32	First 32-bit ALU operand.
operand_b_i	input	32	Second 32-bit ALU operand.
alu_ctrl_i	input	4	Concrete ALU function; encodings are listed in the shared defines.
result_o	output	32	32-bit ALU result for the selected operation.

6.8.2 Functional description

One combinational block computes the selected function over the two operands: addition and subtraction, the bitwise operations, logical and arithmetic shifts by the low five bits of the second operand, and the signed and unsigned comparisons that produce a zero-or-one result.

6.9 rv32i_branch_unit

Resolves branch/jump direction and target; clears JALR target bit 0.

6.9.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
pc_in_i	input	32	PC of the control-transfer instruction in S2.
imm_out_i	input	32	Decoded sign-extended branch/jump displacement.
rs1_data_i	input	32	First forwarded register operand; JALR base or comparison value.
rs2_data_i	input	32	Second forwarded register operand for branch comparison.
opcode_i	input	7	Instruction opcode, bits [6:0].
funct3_i	input	3	Instruction function field; selects operation or memory size/sign.
Branch_i	input	1	Enable conditional-branch evaluation.
Jump_i	input	1	Enable unconditional JAL/JALR evaluation.
branch_target_o	output	32	Resolved branch/jump target; JALR bit 0 is cleared.
pc_src_o	output	1	Next PC takes branch_target_o instead of the sequential address.

6.9.2 Functional description

The comparison of the two register operands is combined with the decoded branch condition, or with an unconditional jump, to produce pc_src_o. The target is the instruction PC plus the displacement, except for JALR, where it is the first operand plus the immediate with bit 0 cleared.

6.10 rv32i_lsu

Performs byte/halfword/word accesses, byte strobes, read extraction and error reporting.

6.10.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
clk_i	input	1	Rising-edge clock input.
rst_n_i	input	1	Asynchronous reset, active low.
req_i	input	1	A valid memory operation is presented this cycle.
we_i	input	1	Access direction: 1 store, 0 load.

Signal	Dir.	Width (bits)	Meaning
funct3_i	input	3	Instruction function field; selects operation or memory size/sign.
addr_i	input	32	Byte address of the access, taken from the ALU result.
st_data_i	input	32	Store data from rs2, after forwarding.
busy_o	output	1	Transaction is not finished; S2 must stall.
done_o	output	1	Response accepted this cycle.
err_o	output	1	Response carried SLVERR or DECERR.
active_o	output	1	Transaction has been issued; an interrupt cannot be accepted mid-access.
ld_data_o	output	32	Load data after size and sign extension; valid on done_o for a read.
dbus_awaddr_o	output	32	Write byte address.
dbus_awprot_o	output	3	Write protection attributes.
dbus_awvalid_o	output	1	Write address is valid; hold until handshake.
dbus_awready_i	input	1	Receiver can accept the write address.
dbus_wdata_o	output	32	Write payload.
dbus_wstrb_o	output	4	One enable bit per write byte.
dbus_wvalid_o	output	1	Write data is valid; hold until handshake.
dbus_wready_i	input	1	Receiver can accept the write data.
dbus_bresp_i	input	2	Write response: OKAY=00, SLVERR=10, DECERR=11.
dbus_bvalid_i	input	1	Write response is valid; hold until handshake.
dbus_bready_o	output	1	Receiver can accept the write response.
dbus_araddr_o	output	32	Read byte address.
dbus_arprot_o	output	3	Read protection attributes.
dbus_arvalid_o	output	1	Read address is valid; hold until handshake.
dbus_arready_i	input	1	Receiver can accept the read address.
dbus_rdata_i	input	32	Read payload.
dbus_rresp_i	input	2	Read response: OKAY=00, SLVERR=10, DECERR=11.
dbus_rvalid_i	input	1	Read response is valid; hold until handshake.
dbus_rready_o	output	1	Receiver can accept the read response.

6.10.2 Functional description

S_IDLE accepts a legal request and selects S_RD or S_WR. S_RD waits for address acceptance and an R handshake; S_WR collects AW and W independently, then waits for B. The response handshake returns the unit to S_IDLE and releases S2. Request address, write data and strobes remain latched during the transaction.

6.11 rv32i_exception_unit

Selects the synchronous trap cause and associated value.

6.11.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
valid_i	input	1	S2 holds a real, non-preempted instruction.
fetch_fault_i	input	1	The fetch of this instruction reported an AXI error.
illegal_i	input	1	Decoder or CSR access reported an illegal operation.
ecall_i	input	1	Current instruction is an ECALL.
ebreak_i	input	1	Current instruction is an EBREAK.
pc_i	input	32	Address of the instruction in S2.
instr_i	input	32	32-bit instruction word to decode or check.
ctl_taken_i	input	1	Resolved control-transfer direction is taken.
ctl_target_i	input	32	Resolved control-transfer target address.
MemRead_i	input	1	Current operation is a load.
MemWrite_i	input	1	Current operation is a store.
funct3_i	input	3	Instruction function field; selects operation or memory size/sign.
mem_addr_i	input	32	ALU result; meaningful only before the access is issued.
mem_active_i	input	1	Data transaction has already been issued.
mem_done_i	input	1	Data response landed this cycle.
mem_err_i	input	1	Data response was SLVERR or DECERR.
exception_o	output	1	A synchronous exception is selected for the current instruction.
cause_o	output	5	Selected synchronous exception code from the cause table.
tval_o	output	32	mtval payload for the selected cause.
mem_misaligned_o	output	1	Address is not naturally aligned; the AXI issue is blocked.

6.11.2 Functional description

Candidate causes are evaluated for the instruction in S2 and one is selected by fixed priority: fetch fault, illegal instruction, environment call or breakpoint, then address misalignment, then a completed data access that returned an error. The selected cause also determines the mtval payload. A misaligned address is reported before the transaction is issued, so a faulting access never reaches the bus.

6.12 rv32i_csr_file

Implements machine-mode control, status and performance counters.

6.12.1 Interface description

Parameter	Default	Meaning / valid setting	
HART_ID	32'd0	Value returned by mhartid.	

Signal	Dir.	Width (bits)	Meaning
clk_i	input	1	Rising-edge clock input.
rst_n_i	input	1	Asynchronous reset, active low.
csr_addr_i	input	12	12-bit CSR address.
csr_wdata_i	input	32	CSR write operand.
csr_op_i	input	2	CSR operation: RW=01, RS=10, RC=11.
csr_ren_i	input	1	CSR read enable.
csr_wen_i	input	1	CSR write enable.
csr_rdata_o	output	32	CSR read value before write.
csr_illegal_o	output	1	Unimplemented address or disallowed write.
trap_set_i	input	1	Commit a trap entry.
trap_is_irq_i	input	1	Trap is an interrupt.
trap_code_i	input	5	Trap cause code.
trap_pc_i	input	32	PC to save in mepc.
trap_val_i	input	32	Value to save in mtval.
mret_i	input	1	Commit trap return.
irq_lines_i	input	16	Pending interrupt sources.
irq_enable_o	output	16	Enabled interrupt sources.
mie_global_o	output	1	Current value of mstatus.MIE.
trap_vector_o	output	32	Direct or vectored trap address.
mepc_out_o	output	32	Saved aligned return PC.
retire_i	input	1	One non-trapping instruction completes S2.
ev_mispredict_i	input	1	Branch misprediction event.
ev_ibus_wait_i	input	1	S2 lacks a valid instruction.
ev_dbus_stall_i	input	1	Data transaction active.
ev_wfi_sleep_i	input	1	WFI sleep cycle.

6.12.2 Functional description

A CSR instruction reads the pre-write value. RW replaces bits, RS sets bits and RC clears bits; immediate forms zero-extend the five-bit operand. Trap entry has priority over software writes. Counter writes take priority over increment and preserve the unaddressed half.

6.13 rv32i_hazard_unit

Controls advance/flush and detects S3-to-S2 forwarding; an active LSU keeps S2 stalled.

6.13.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
fetch_valid_i	input	1	S1 offers an instruction this cycle.
lsu_busy_i	input	1	A data transaction is still in flight in S2.
wfi_wait_i	input	1	WFI is in S2 and no wake condition exists yet.
trap_take_i	input	1	A synchronous exception or an accepted interrupt commits this cycle.
mret_exec_i	input	1	A valid MRET instruction is executing.
mispredict_i	input	1	Resolved direction or target differs from the prediction.
rs1_i	input	5	First source GPR index (or immediate CSR operand).
rs2_i	input	5	Second source GPR index.
wb_valid_i	input	1	S3 contains a valid instruction.
wb_reg_write_i	input	1	S3 instruction enables a GPR write.
wb_rd_i	input	5	S3 destination GPR index.
s2_advance_o	output	1	S2 finishes its instruction this cycle.
redirect_o	output	1	Fetch changes path; held S1 content is dropped.
if_dx_we_o	output	1	IF/DX loads a new instruction or a bubble.
if_dx_bubble_o	output	1	The value loaded into IF/DX is a bubble.
dx_squash_o	output	1	The S2 instruction must not commit to S3.
fwd_rs1_o	output	1	Use S3 writeback data as the first S2 register operand.
fwd_rs2_o	output	1	Use S3 writeback data as the second S2 register operand.

6.13.2 Functional description

The unit forms s2_advance from LSU/WFI blocking conditions. An advancing redirect inserts an IF/DX bubble; trap entry additionally suppresses S2 writeback. Forwarding matches the valid S3 destination against each S2 source, excluding x0.

6.14 rv32i_writeback_mux

Selects ALU/CSR result, loaded data or PC+4 for register writeback.

6.14.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
alu_result_i	input	32	ALU result, or the CSR read value muxed in S2.
mem_data_i	input	32	Completed load value after size/sign extraction.
pc_plus4_i	input	32	Sequential next PC, used as the JAL/JALR link value.
MemtoReg_i	input	2	Writeback source: 00 ALU/CSR; 01 load; 10 PC+4.
wb_data_o	output	32	Selected 32-bit value to write to the destination GPR.

6.14.2 Functional description

The select encoding chooses among the ALU or CSR result, the extended load data and the sequential PC+4 link value. The chosen value is what S3 writes to the destination register and what the forwarding path returns to S2.

7 List of bugs and workarounds

No open RTL defects are known at revision 2.2. Limitations that software and follow-up projects have to work around:

Limitation	Workaround
FENCE.I is not implemented and traps as an illegal instruction.	Load instruction memory before the core executes from it; code cannot modify itself.
Misaligned loads and stores trap (cause 4 / 6).	Align the data, or emulate the access in the trap handler.
The user counters cycle, time and instret (0xC00–0xC02) and mcountinhibit (0x320) are absent and trap.	Read mcycle and minstret in machine mode.
The trap-kind stack holds 16 levels, interrupts and exceptions combined.	Keep the combined nesting depth at or below 16, for example through NEST_MAX.

8 Annex

8.1 Shared Verilog definitions

These macros are defined once in `rv32i_defines.vh` and used by several modules. They are listed so that an encoding named in a port table can be resolved without opening the source.

Definition	Value	Use
OPC_LUI	7'b0110111	Instruction opcode
OPC_AUIPC	7'b0010111	Instruction opcode
OPC_JAL	7'b1101111	Instruction opcode
OPC_JALR	7'b1100111	Instruction opcode
OPC_BRANCH	7'b1100011	Instruction opcode
OPC_LOAD	7'b0000011	Instruction opcode
OPC_STORE	7'b0100011	Instruction opcode
OPC_OP_IMM	7'b0010011	Instruction opcode
OPC_OP	7'b0110011	Instruction opcode
OPC_FENCE	7'b0001111	Instruction opcode
OPC_SYSTEM	7'b1110011	Instruction opcode
ALUOP_ADD	4'b0000	ALU decode
ALUOP_FUNCT	4'b0010	ALU decode
ALUOP_LUI	4'b0011	ALU decode
ALUOP_AUIPC	4'b0100	ALU decode
ALU_ADD	4'b0000	ALU operation
ALU_SUB	4'b0001	ALU operation
ALU_AND	4'b0010	ALU operation
ALU_OR	4'b0011	ALU operation
ALU_XOR	4'b0100	ALU operation
ALU_SLL	4'b0101	ALU operation
ALU_SRL	4'b0110	ALU operation
ALU_SRA	4'b0111	ALU operation
ALU_SLT	4'b1000	ALU operation
ALU_SLTU	4'b1001	ALU operation
ALU_PASSB	4'b1010	ALU operation
WB_ALU	2'b00	Writeback source
WB_MEM	2'b01	Writeback source
WB_PC4	2'b10	Writeback source
CSROP_RW	2'b01	CSR operation
CSROP_RS	2'b10	CSR operation
CSROP_RC	2'b11	CSR operation
CAUSE_IMISALIGN	5'd0	Trap cause

Definition	Value	Use
CAUSE_IFAULT	5'd1	Trap cause
CAUSE_ILLEGAL	5'd2	Trap cause
CAUSE_BREAK	5'd3	Trap cause
CAUSE_LD_MISALIGN	5'd4	Trap cause
CAUSE_LD_FAULT	5'd5	Trap cause
CAUSE_ST_MISALIGN	5'd6	Trap cause
CAUSE_ST_FAULT	5'd7	Trap cause
CAUSE_ECALL_M	5'd11	Trap cause